Module 10: Aspect Oriented Programming (AOP) in Spring Boot

Questions

Q1. What is Aspect Oriented Programming, and why is it useful in a Spring Boot application?

Answer: Aspect Oriented Programming is a programming paradigm that separates cross-cutting concerns from the main business logic. Cross-cutting concerns are requirements that cut across many modules, such as logging, transaction management, security, auditing, and caching. Without AOP, every service method would be cluttered with repetitive code for these concerns. AOP lets you define the concern once in an aspect and apply it declaratively to many join points. In a Spring Boot application, this makes the business code cleaner and easier to maintain, because the infrastructure behavior is woven in by the framework instead of being copied into every method.

Q2. What are the core concepts of Spring AOP: aspect, join point, pointcut, advice, and weaving?

Answer: An aspect is a modular unit that encapsulates a cross-cutting concern. A join point is a specific point in program execution where the aspect can be applied, such as a method execution or exception being thrown. A pointcut is an expression that selects which join points the advice applies to. Advice is the action taken by the aspect at a particular join point, such as logging before a method runs. Weaving is the process of applying aspects to the target objects to create an advised object. In Spring AOP, weaving happens at runtime using proxies, which is simpler than compile-time or load-time weaving but limited to method-level join points on Spring beans.

Q3. What types of advice does Spring AOP support?

Answer: Spring AOP supports five types of advice. `@Before` runs before the method execution. `@AfterReturning` runs after a method completes normally and can access the return value. `@AfterThrowing` runs if the method throws an exception and can access the exception. `@After` runs after the method regardless of outcome, similar to a finally block. `@Around` wraps the method execution entirely, letting the aspect decide whether to proceed, modify arguments, modify the return value, or catch and handle exceptions. `@Around` is the most powerful advice because it controls the full execution flow, but it is also the easiest to misuse because forgetting to call `proceed()` silently skips the target method.

Q4. What is a pointcut expression, and how do you write one in Spring AOP?

Answer: A pointcut expression selects join points using the AspectJ pointcut expression language.

The `execution()` designator is the most common, for example `execution(* com.example.service.*.*(..))` matches every method in the service package. You can narrow by return type, package, class, method name, and parameter types. Other designators include `within()` for matching types, `this()` and `target()` for matching proxy or target types, and `@annotation()` for matching methods that have a specific annotation. Pointcut expressions can be combined with `&&`, `||`, and `!`. A poorly written broad pointcut can accidentally intercept too many methods and hurt performance or change behavior unexpectedly.

Q5. How do you declare a pointcut using `@Pointcut`?

Answer: You declare a reusable pointcut by annotating an empty method with `@Pointcut` and providing the expression. For example, `@Pointcut("execution(* com.example.service.*.*(..))") public void serviceLayer() {}`. Other advice methods can then reference this named pointcut, such as `@Before("serviceLayer()")`. This makes the aspect easier to read and maintain because the complex expression is written once and reused. It also makes it easy to change the matching logic in one place. The method that carries `@Pointcut` has no runtime behavior; it exists only to hold the expression.

Q6. What is the difference between `@Pointcut("execution(...)")` and `@Pointcut("@annotation(...)")`?

Answer: `execution(...)` matches methods based on their signature, such as package, class, method name, and parameters. It is useful when you want to apply advice to an entire layer, like all methods in the service package. `@annotation(...)` matches methods based on whether they are annotated with a specific annotation. This is useful when you want to opt methods in individually, such as applying a custom `@LogExecutionTime` annotation only to selected methods. The annotation-based approach is more explicit and less likely to accidentally match new methods added later, while the execution-based approach is better for layer-wide concerns.

Q7. What is an `@Around` advice, and how do you write it?

Answer: `@Around` advice wraps a method execution and is declared on a method that takes a `ProceedingJoinPoint` parameter. Inside the advice you can run code before and after `joinPoint.proceed()`, decide not to call proceed at all, catch exceptions, or change the return value. For example, an `@Around` advice can start a timer, call the method, then log how long it took. The advice must return an object compatible with the target method's return type. A critical mistake is forgetting to return the result of `proceed()`, which makes every intercepted method return `null`. Another mistake is swallowing exceptions inside the advice without rethrowing them, which hides failures from the caller.

Q8. How does `@Before` advice differ from `@AfterReturning` advice?

Answer: `@Before` advice runs before the target method starts and cannot prevent the method from running except by throwing an exception. It is useful for validation, logging inputs, or setting context. `@AfterReturning` advice runs only if the method completes successfully and gives access to the return value through the `returning` attribute. It is useful for post-processing results, caching return values, or logging successful outcomes. If the method throws an exception, `@AfterReturning` does not run, while `@After` or `@AfterThrowing` would be appropriate. Choosing the wrong advice type can lead to missing error handling or running cleanup too early.

Q9. What happens if two aspects apply to the same join point, and how do you control their order?

Answer: When multiple aspects advise the same method, Spring executes them in a specific order. By default the order is undefined unless the aspects implement the `Ordered` interface or are annotated with `@Order`. Lower order values run first, so an aspect with `@Order(1)` runs before an aspect with `@Order(2)`. For `@Around` advice, the outermost aspect starts first, calls the next aspect, and finishes last, like nested method calls. If the order matters for correctness, for example a security aspect must run before a logging aspect, you should explicitly define `@Order` or implement `Ordered`. Without ordering, behavior can appear random and hard to reproduce.

Q10. What is a join point signature, and how do you access method information inside advice?

Answer: The join point signature is the metadata about the method being intercepted. In advice methods you can inject a `JoinPoint` parameter and call methods like `getSignature()`, `getArgs()`, `getTarget()`, and `getThis()`. The signature gives you the method name, declaring type, return type, and parameter types. This is useful for writing generic logging or auditing aspects that do not need to know the exact method in advance. For example, `Signature signature = joinPoint.getSignature(); String methodName = signature.getName();` lets you log which method was called and with what arguments. Be careful not to log sensitive arguments such as passwords or tokens in a generic aspect.

Q11. What is the difference between `this()` and `target()` in pointcut expressions?

Answer: In Spring AOP, `this()` refers to the proxy object, and `target()` refers to the actual target object behind the proxy. Because Spring AOP creates a proxy around the bean, `this` and `target` are usually different references unless the proxy is not used. In practice, `this()` is rarely needed, while `target()` is used when you want to match methods based on the actual implementation type rather than the interface. For most developers, `execution()` and `@annotation()` cover the majority of use cases, and `this()` and `target()` are advanced tools used in specialized scenarios.

Q12. What are the limitations of Spring AOP compared to AspectJ?

Answer: Spring AOP is proxy-based and works only on Spring beans. It supports method-level join points but not field access, constructor execution, or static initializer join points. AspectJ, the full framework, supports all join points through compile-time or load-time weaving. Spring AOP is easier to configure and sufficient for most enterprise concerns like transactions and logging. AspectJ is more powerful but adds complexity to the build process. If you need advice on field access or non-Spring objects, AspectJ is the right choice. For typical Spring Boot applications, Spring AOP handles the common cross-cutting concerns without needing AspectJ.

Q13. How does Spring AOP create proxies, and what is the difference between JDK dynamic proxies and CGLIB?

Answer: Spring AOP creates proxies to wrap target beans and apply advice. If the target class implements at least one interface, Spring uses a JDK dynamic proxy that implements the same interfaces. If the target class does not implement an interface, Spring uses CGLIB to generate a subclass of the target. As of Spring Boot 2 and later, CGLIB is used even for interface-based targets if `spring.aop.proxy-target-class` is set to `true`, which is the default. This matters because proxy-based AOP cannot intercept calls made from within the same object, so self-invocation bypasses the aspect. Understanding the proxy mechanism is essential for debugging cases where advice does not run.

Q14. What is the self-invocation problem in Spring AOP, and how do you avoid it?

Answer: The self-invocation problem happens when a method in a bean calls another method on the same bean, and the second call bypasses the Spring proxy. Because Spring AOP applies advice through the proxy, internal calls do not go through the proxy and therefore do not trigger aspects. This commonly breaks `@Transactional`, `@Cacheable`, and `@PreAuthorize`. To avoid it, you can inject the same bean into itself through Spring, use `AopContext.currentProxy()` to obtain the current proxy, or refactor the logic so the called method resides in a separate bean. The cleanest solution is usually to split the responsibilities into separate Spring-managed beans.

Q15. How do you enable and configure AOP in a Spring Boot application?

Answer: Spring Boot auto-configures AOP when it detects `spring-boot-starter-aop` or `spring-aop` and AspectJ Weaver on the classpath. You create an aspect class with `@Aspect` and register it as a Spring bean, usually by adding `@Component` or by declaring it in a `@Configuration` class. If you are not using component scanning, you can import the aspect with `@Import`. The main application or a configuration class does not need `@EnableAspectJAutoProxy` in Spring Boot because the starter enables it automatically, but

you can add it if you want to customize settings such as `proxyTargetClass` or `exposeProxy`. A common issue is forgetting to make the aspect a bean, which means it is never picked up by Spring.

Q16. What are some real-world use cases for AOP in Spring Boot?

Answer: Common real-world use cases include logging method entry and exit, measuring execution time for performance monitoring, retrying transient failures, auditing who changed what and when, applying declarative transaction management, caching results with `@Cacheable`, and enforcing security checks. AOP is also used for rate limiting, exception translation, and metrics collection. The value is that the business code stays focused on business rules while the infrastructure concern is applied transparently. However, AOP should be used for concerns that are truly cross-cutting; overusing it for one-off behavior makes code harder to trace.

Q17. How would you use AOP to log method execution time?

Answer: I create an aspect with a pointcut that matches the methods I want to measure, and an `@Around` advice that records the time before and after calling `proceed()`. For example, `@Around("@annotation(com.example.annotation.LogExecutionTime)")` lets me annotate specific methods, and the advice computes `System.currentTimeMillis()` before and after, then logs the difference. This avoids adding timing code to every method manually. I would also include the class and method name from the join point signature so the log is useful. I am careful not to log execution time for extremely high-frequency methods because the logging itself can become a bottleneck.

Q18. How do you use AOP for audit logging in a Spring Boot application?

Answer: I define an `@Audit` custom annotation and an aspect with an `@AfterReturning` or `@Around` advice that matches `@annotation(com.example.annotation.Audit)`. The advice reads the authenticated user from `SecurityContextHolder`, the method name and arguments from the join point, the timestamp, and the outcome, and writes a record to an audit table or sends it to an audit service. This keeps every audited method clean because the annotation is the only marker needed. I avoid logging sensitive fields from method arguments and make the audit writer asynchronous if it involves an external system, so it does not block the main request.

Q19. Can AOP be used for security, and what are the trade-offs?

Answer: AOP can enforce security checks such as verifying roles, validating API keys, or checking ownership before method execution. A custom aspect can intercept service methods and reject unauthorized calls with an exception. The trade-off is that AOP security is method-level and can be bypassed by self-invocation, so it is not as robust as HTTP-layer security for web endpoints. For most cases, Spring Security's URL and method security are the better choice because they are well

tested and handle many edge cases. AOP security is useful for custom domain-level checks that are not covered by Spring Security expressions, such as verifying that a user owns a specific resource before allowing deletion.

Q20. How do you test code that uses Spring AOP aspects?

Answer: For unit tests of the business logic, I test the target class directly without Spring, so the aspect is not involved. For integration tests, I load the Spring context and verify that the aspect is applied correctly, for example by checking that a logging aspect wrote a log entry or that a transaction rolled back on an exception. I use `@SpringBootTest` or a sliced test with the aspect imported through `@Import`. When testing `@Around` advice, I pay special attention to the case where `proceed()` is or is not called, and to exception propagation. I also test self-invocation scenarios to confirm that advice does not run when it is expected not to, because that is a common production surprise.

Q21. What is the `ProceedingJoinPoint` interface, and when is it used?

Answer: `ProceedingJoinPoint` is a subtype of `JoinPoint` available only in `@Around` advice. It extends `JoinPoint` with the `proceed()` method, which triggers the actual method execution. You can call `proceed()` once, or not at all, or even call it multiple times in special cases such as retry logic. You can also pass modified arguments to `proceed(Object[] args)` if you need to change the parameters passed to the target method. Because `ProceedingJoinPoint` controls the flow, misuse such as forgetting to call `proceed()` or swallowing exceptions can have wide impact across all intercepted methods, so it should be reviewed carefully.

Q22. What is the difference between `@After` and `@AfterReturning` advice?

Answer: `@After` advice runs after the method completes no matter what, whether it returns normally or throws an exception. It is typically used for cleanup, such as closing resources or releasing locks. `@AfterReturning` advice runs only when the method returns normally, and it can capture and inspect the return value. It is useful for post-processing successful results, such as transforming the return object or caching it. If you need both the return value and error handling, you should use `@Around` advice or a combination of `@AfterReturning` and `@AfterThrowing`. Using `@After` when you actually need the return value is a common mistake.

Q23. What is load-time weaving, and when would you use it with Spring?

Answer: Load-time weaving is a technique where AspectJ instruments classes as they are loaded by

the classloader, allowing advice on a broader set of join points than Spring AOP's runtime proxies can handle. It requires a Java agent or a special classloader and is configured with `@EnableLoadTimeWeaving`. You would use it when you need aspects to apply to classes that are not Spring beans, to constructor execution, or to field access. It adds complexity to deployment because the agent must be attached to the JVM, so it should only be used when Spring AOP is genuinely insufficient. Most Spring Boot applications do not need load-time weaving.

Q24. How do you avoid performance problems when using AOP heavily?

Answer: The main performance risks come from broad pointcuts that match too many methods and from expensive advice such as heavy logging, synchronous external calls, or reflection. I keep pointcuts as specific as possible, using package names or custom annotations rather than wildcard patterns that match the entire application. I avoid logging large objects or serializing arguments in the advice. For heavy work like sending audit events, I offload it to an asynchronous mechanism. I also measure the overhead of the aspect in production-like load tests. AOP is powerful, but invisible overhead can accumulate across many intercepted methods, so it should be reviewed during code reviews and profiling.

Scenario based questions

S1. A logging aspect is supposed to log every service method call, but some calls are missing from the logs. How do you investigate?

Answer: I start by checking the pointcut expression. If the pointcut is written too narrowly, for example matching only one package or excluding methods with certain parameter types, it will miss methods added outside that scope. I also verify that the affected bean is actually a Spring bean, because AOP works only on beans managed by the application context. The most common cause is self-invocation: a method inside the bean calls another method on the same instance, bypassing the proxy entirely. I confirm this by adding a small test that calls the method from outside the bean and from inside the bean, and comparing which calls trigger the advice. If self-invocation is the cause, I refactor the code so the internal call goes through a separate bean, or I use `AopContext.currentProxy()` if the method must stay in the same class. I also make sure the aspect class is registered as a Spring bean; an `@Aspect` class without `@Component` or an explicit bean declaration is ignored.

S2. A transaction rollback aspect works for direct service calls but fails when one service method calls another method in the same class. Why?

Answer: This is the self-invocation problem. Spring applies transactions through a proxy, so the `@Transactional` advice runs only when a caller outside the bean invokes the method through the proxy. When a method inside the same bean calls another method, the call happens on the raw target object, not the proxy, so the transaction advice is bypassed. To fix it, I move the called

method into a separate Spring bean and inject it, which ensures every call goes through a proxy. Alternatively, I can inject the same bean into itself using `@Autowired`, or use `AopContext.currentProxy()` to obtain the current proxy and invoke the method through it. The cleanest long-term fix is better separation of responsibilities so that self-invocation is not needed.

S3. An `@Around` advice for retry logic causes an infinite loop. What is the likely cause?

Answer: An infinite loop in a retry aspect usually means the advice catches an exception and retries, but the retry call itself is being re-advised by the same aspect, which catches the same exception again and retries indefinitely. I fix this by limiting retries with a counter or by excluding the retry path from the pointcut, for example by checking a thread-local flag that indicates the call is already inside a retry. Another cause is retrying on exceptions that are never going to succeed, such as validation or authentication failures, so I make the retry logic selective about which exceptions warrant a retry. I also add exponential backoff and a maximum retry count, and I log each attempt so it is clear what is happening.

S4. A performance timing aspect makes the application noticeably slower. How do you optimize it?

Answer: I first check the pointcut to see whether it matches too many methods, including high-frequency utility methods or getters. I narrow the pointcut, ideally using a custom annotation so only selected methods are timed. I also look at what the advice does: logging every method call with full argument serialization is expensive, especially for large objects or collections. I reduce the logging to just class name, method name, and duration in milliseconds. If the aspect performs synchronous I/O, such as writing to a database or sending metrics to an external system, I move that work to an asynchronous executor. I run a load test with and without the aspect to measure the overhead and decide whether the value of the timing data is worth the cost.

S5. Two aspects, one for logging and one for security, are running in the wrong order. How do you fix it?

Answer: I implement the `Ordered` interface or add `@Order` to both aspects to define a clear execution order. Lower values run first, so I give the security aspect `@Order(1)` and the logging aspect `@Order(2)` if security must be checked before logging. For `@Around` advice, the order also affects which aspect wraps the other; the outermost advice starts first and finishes last. I avoid relying on the default order because it is not guaranteed and can change when new aspects are added. After setting the order, I write an integration test that exercises both aspects on the same method and verifies the expected sequence, for example by checking that a security exception is thrown before a log entry is written.

S6. An auditing aspect logs sensitive information such as passwords. How do you fix it?

Answer: I update the auditing aspect to inspect the join point arguments and redact or skip fields marked as sensitive. This can be done by checking for annotations such as `@Sensitive` or `@JsonIgnore` on fields, or by maintaining a deny list of field names like `password`, `token`, and `ssn`. If the aspect currently serializes the entire argument object with `toString()` or Jackson, I replace that with a sanitized view. I also add a code review rule that any new audit aspect must be reviewed for sensitive data exposure. In some cases, the safest approach is to log only the action, the entity identifier, and the user identity, without logging the request payload at all.

S7. A caching aspect using `@Around` returns stale data because it does not invalidate the cache correctly. How do you debug it?

Answer: I start by confirming whether the cache key is being built from the right method arguments and whether the same arguments produce the same key for both read and write operations. If the key includes a timestamp or a mutable object, it can change unexpectedly between calls. I check the `@Around` advice to make sure it stores the result in the cache after `proceed()` and returns the cached value on subsequent calls before calling `proceed()`. I also verify that the invalidation logic runs for update and delete methods, and that the cache key used for invalidation matches the key used for reads. A subtle issue is catching exceptions in the advice and returning a cached fallback; that can hide errors and serve stale data, so I make sure the fallback behavior is intentional and documented.

S8. A custom security aspect works for most methods but is bypassed when called from a test. Why?

Answer: Tests that instantiate the service class directly with `new` do not go through the Spring proxy, so no AOP advice is applied. This is actually correct behavior, because the test is exercising the business logic without the infrastructure. If the test needs to verify the security aspect, it should load the Spring context and obtain the bean from the context, or use `@SpringBootTest` with the aspect imported. Another possibility is that the test uses `@MockBean` for a dependency and bypasses the real service entirely. I make sure the test suite has two kinds of tests: pure unit tests that do not involve AOP, and integration tests that verify AOP behavior through the proxy.

S9. A new developer adds a method to a service, and the logging aspect stops working for that method. What happened?

Answer: The most likely cause is that the pointcut expression does not match the new method. For example, if the pointcut matches `execution(* com.example.service.*.*(..))` and the new method is in a subpackage or returns a type that is excluded by the expression, it will not be intercepted. Another cause is that the new method is `private` or `final`, which cannot be proxied by CGLIB in some cases. I review the pointcut expression and the location of the new method, and I add the method to the matching scope or use a custom annotation to opt it in. I also consider

whether the method is called internally, which would bypass the proxy through self-invocation. A good practice is to keep pointcut expressions broad enough for the intended layer but specific enough to avoid surprises.

S10. The application uses AOP for exception translation, but exceptions are swallowed and never reach the controller. How do you fix it?

Answer: In an `@Around` advice that translates exceptions, the aspect probably catches the checked exception and returns a fallback or null instead of rethrowing a runtime exception. The fix is to catch the original exception and throw a meaningful unchecked exception that the controller's global exception handler can convert into a proper HTTP response. If the aspect is meant to translate a low-level exception such as `DataAccessException` into a domain exception, it should throw the new exception and let it propagate. I also make sure the pointcut does not match the exception handler itself, which could create a loop. Logging the original exception before translation is important so the root cause is preserved in the logs.

S11. A rate-limiting aspect is added, but it blocks legitimate traffic during bursts. How do you tune it?

Answer: I first examine the rate limit parameters: the maximum number of requests, the time window, and whether the limit is global or per user. A global limit is easy to set too low for traffic spikes. I switch to per-user or per-API-key limits if the goal is to prevent abuse without hurting all users. I also consider using a token bucket or leaky bucket algorithm instead of a fixed window, because they handle bursts more gracefully. The aspect should return a clear 429 Too Many Requests response with a `Retry-After` header. I add metrics to track how often the limit is hit, and I tune the thresholds based on real traffic patterns rather than guesses. If the limit is backed by a shared store like Redis, I also check for latency from that store.

S12. A distributed tracing aspect adds a trace ID to outgoing HTTP calls, but the ID is missing in some downstream services. Why?

Answer: The trace ID is probably being set in the aspect but not propagated consistently. If the downstream call is made inside a new thread or an async method, the trace ID stored in thread-local MDC may not be copied to the new thread. I fix this by using `TaskDecorator` to copy MDC values when tasks are submitted to an executor, or by using a context propagation library. Another cause is that the aspect intercepts only synchronous methods and misses calls made through `RestTemplate`, `WebClient`, or a Feign client that builds its own headers. In that case, I use an interceptor specific to the HTTP client instead of, or in addition to, the AOP aspect. I verify propagation with an integration test that checks headers across service boundaries.

S13. An aspect that sends audit events to Kafka is causing requests to slow down. How do you make it non-blocking?

Answer: I move the Kafka producer call out of the request thread and into an asynchronous mechanism. The simplest approach is to wrap the audit writer in a method annotated with `@Async` and backed by a `TaskExecutor`. The aspect can then call the async writer after the main method completes, or use an `ApplicationEventPublisher` to fire an event and handle it asynchronously with `@EventListener`. A more robust approach is to write the audit event to a local queue or buffer and have a background worker batch-send events to Kafka. I also make the producer fire-and-forget for non-critical audit data, or use a separate thread pool with proper backpressure and error handling so the main request is never blocked by Kafka slowness.

S14. A pointcut meant to match repository methods also matches internal Spring Data methods, causing unexpected behavior. How do you narrow it?

Answer: I narrow the pointcut by excluding Spring Data internal methods using `execution(* com.example.repository.*.*(..)) && !execution(* org.springframework.data.repository.*.*(..))` or by targeting only specific custom repository interfaces. Spring Data generates many implementation methods at runtime, and broad pointcuts can match them. A better approach is to use a custom annotation such as `@Audit` on the repository methods that actually need advice, which avoids matching generated methods entirely. I also use `within()` to restrict matching to a specific interface or package. After changing the pointcut, I run a test that lists all matched methods to confirm that only the intended ones are intercepted.

S15. A method is annotated with `@Transactional` and a custom `@Audit` aspect, but the audit record is written even when the transaction rolls back. How do you keep them in sync?

Answer: The audit advice runs independently of the transaction, so it writes the record regardless of the transaction outcome. To keep them in sync, I change the audit advice to participate in the transaction. One approach is to write the audit record through the same transaction by using `@Transactional` on the audit writer or by calling it from within the same service transaction boundary. Another approach is to publish an application event after the transaction commits successfully, using `@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)`. This ensures the audit is recorded only when the transaction commits. If the audit must be independent of the database for reliability, I use a transactional outbox pattern: write the audit event to an outbox table in the same transaction, and have a background process publish it.

S16. A `@Before` advice is validating input, but the validation errors are not returned as proper API responses. How do you fix it?

Answer: The `@Before` advice throws an exception, but the exception type is not mapped by the global exception handler, so the controller returns a generic 500 response instead of a structured 400 response. I create a dedicated domain exception such as `InvalidInputException` and throw it from the advice. Then I add a `@ExceptionHandler` or `@RestControllerAdvice` method that catches the exception and returns a `ResponseEntity` with a clear error message, field details, and HTTP 400 status. I also make sure the validation advice does not match the exception handler itself, which could intercept the response and alter it. For complex validation, I prefer moving the rules into the service layer or using Bean Validation with `@Valid`, because validation errors are easier to map to HTTP responses there.

S17. An aspect is supposed to run only in production, but it also runs during unit tests and breaks them. How do you control this?

Answer: I make the aspect conditional on a Spring profile using `@Profile("prod")` or by making it active only when a specific property is set. In the test configuration, I exclude the aspect from component scanning or override the aspect bean with a no-op version. For pure unit tests that instantiate services directly, the aspect is not involved anyway, but for integration tests that load the full context, profile-based activation is the cleanest solution. I also consider whether the aspect truly needs to run in tests; if not, I add a test profile that disables it. Another option is to use `@ConditionalOnProperty` on the aspect bean so it is created only when `app.audit.enabled=true`, and keep it false in tests.

S18. A new `@Around` advice is added for request validation, and now some methods return null unexpectedly. Why?

Answer: The most likely cause is that the `@Around` advice does not return the result of `proceed()`. If the advice returns a hardcoded value, a default value, or nothing at all, the caller receives that value instead of the real method result. I check the advice method to ensure it stores the return value of `joinPoint.proceed()` and returns it to the caller. Another cause is that the advice swallows an exception and returns null instead of rethrowing it. I add unit tests for the aspect itself and integration tests that verify the intercepted methods still return their expected values and throw their expected exceptions when the validation passes.

S19. An aspect collects metrics using Micrometer, but the metrics are not tagged with the method name. How do you improve the metric?

Answer: I update the aspect to extract the method name and class name from the join point signature and add them as tags on the Micrometer timer or counter. For example, `Timer.builder("method.execution").tag("class", className).tag("method",`

`methodName).register(meterRegistry).record(duration)` gives a breakdown per method. I am careful not to create an unbounded number of tags, because each unique combination of tag values creates a new time series in the monitoring backend. For methods with high cardinality arguments, I tag only the class and method, not the argument values. I also add a common tag for the application name and environment so metrics can be filtered in dashboards.

S20. A developer wants to use AOP to modify the return value of every controller method to wrap it in a common envelope. What are the risks?

Answer: Wrapping return values globally with AOP is risky because it changes the contract that controllers expose to clients and other internal code. It can break existing tests, swagger generation, client SDKs, and response handling in the frontend. It also makes the response format inconsistent if some endpoints bypass the aspect, such as error responses from `@RestControllerAdvice`. A cleaner approach is to use a `ResponseBodyAdvice` implementation, which is designed specifically for transforming controller responses and integrates better with Spring MVC's message conversion. If AOP is used anyway, the pointcut must be carefully limited to the exact controller package, and the advice must handle void methods, exception paths, and streaming responses correctly.

S21. A caching aspect with `@Around` advice caches the result of a method that returns a `Flux` or `Stream`, and the caller gets a closed stream on the second invocation. How do you fix it?

Answer: This happens because reactive types and Java streams can be consumed only once. The first call consumes the `Flux` or `Stream` and stores the exhausted result in the cache, so the next caller receives an already-consumed or closed object. To fix it, I change the design so the aspect does not cache the raw reactive type. For reactive code, I move caching into the reactive pipeline using operators or reactive cache libraries that understand backpressure and subscription. For streams, I collect the results into a list before caching if the dataset is small, or I redesign the method to return a repeatable data structure such as `List`. The deeper lesson is that AOP advice must understand the return type semantics; wrapping any return value blindly can break lazy or single-use types.

S22. A logging aspect works perfectly in a local Spring Boot test but logs nothing in production. What could be different?

Answer: The first thing I check is whether the aspect class is actually a bean in the production context. If the aspect is in a package that is not covered by component scanning in production, or if it is conditional on a profile or property that is not active, it will not be registered. I also verify that the production pointcut matches the real package structure, because production packages may differ from test packages. Another possibility is that the production application uses a different proxy mode, such as `spring.aop.proxy-target-class=false`, which causes JDK dynamic proxies to be used for interface-based beans; if the aspect pointcut relies on the target class rather

than the interface, it may not match. I add a startup bean that lists all aspects in the context and logs them, which helps verify whether the aspect is present.

S23. A transaction timing aspect measures a `@Transactional` method and reports that it always completes in under 10 milliseconds, even though the database query takes several seconds. What is wrong?

Answer: The aspect is probably measuring only the time taken by the proxy dispatch, not the actual transaction. This can happen if the advice intercepts a method on the proxy but the real transactional work happens on a different thread or inside a lazy-loaded operation that executes after the method returns. Another common cause is that the aspect measures the outer method but not the inner `@Transactional` method, because self-invocation bypassed the transaction proxy. To fix it, I verify where the advice is attached and whether it uses `joinPoint.proceed()` correctly. For accurate measurement, I use Micrometer timers around the actual repository or service call, or I rely on Hibernate statistics and database metrics rather than a generic AOP timer that may miss the real work.

S24. A team adds AOP to enforce a cross-cutting rule, but debugging becomes difficult because the behavior is invisible in the code. How do you maintain readability?

Answer: The invisible nature of AOP is both its strength and its risk. I maintain readability by using custom annotations as markers for the cross-cutting concern, so a developer can see that a method is `@Audited`, `@Retryable`, or `@Timed` just by reading the business code. I keep the aspects themselves small and focused, with clear names that describe the concern. I document the aspects in the project README or developer guide and list which packages or annotations they apply to. I avoid broad pointcuts that apply to half the application, because those are hard to discover. During code reviews, I treat aspects as first-class code and require the same test coverage as services. If a behavior is not truly cross-cutting, I prefer explicit code in the method over hidden magic.

S25. An `@AfterThrowing` advice is supposed to send an alert when a service method fails, but it fires for business exceptions that are expected. How do you refine it?

Answer: The pointcut is too broad and matches every exception, including business validation errors that are normal outcomes. I refine the `@AfterThrowing` annotation with the `throwing` and optional `argNames` attributes, and inside the advice I check the exception type. I either rethrow only specific unchecked or system exceptions, or I configure the pointcut to exclude known business exception types. For example, I can define a marker interface such as `BusinessException` and only send alerts for exceptions that are not assignable to it. Another approach is to handle business exceptions in the controller layer and let only unexpected exceptions propagate to the aspect. This prevents alert fatigue and keeps the on-call team focused on real

problems.

S26. A method annotated with both `@Async` and a custom AOP annotation behaves inconsistently: sometimes the aspect runs, sometimes it does not. Why?

Answer: The behavior depends on which proxy handles which concern. Spring creates one proxy per bean, and a single proxy can handle multiple advisors, but the order and composition matter. If `@Async` is handled by one proxy and the custom aspect by another, or if one annotation is processed before the proxy is fully created, the result can be inconsistent. I verify that both concerns are applied to the same proxy by checking the bean at runtime with `AopUtils.isAopProxy()` and `AopUtils.isCglibProxy()`. I also ensure that the aspect and the async executor are configured to work together, for example by making the aspect run before `@Async` so it measures or audits the actual method invocation. If the interactions are too complex, I move one of the concerns into explicit code or a dedicated wrapper service to make the behavior deterministic.

S27. A pointcut that matches all public methods in the service layer also intercepts methods called from `@Scheduled` tasks, causing duplicate audit entries. How do you fix it?

Answer: Scheduled tasks run through Spring's task scheduler, which also creates proxies, so the same pointcut can match methods invoked from scheduled tasks as well as from HTTP requests. To fix it, I refine the pointcut to exclude scheduled methods, for example by adding `!@annotation(org.springframework.scheduling.annotation.Scheduled)` to the expression. Alternatively, I use a custom annotation that is placed only on the methods that should be audited, rather than matching the entire service layer. If scheduled tasks need different audit behavior, I create a separate aspect or a separate annotation for them. The broader fix is to design pointcuts around intent, not just location, so the same concern can be applied differently in different invocation contexts.

S28. An aspect that modifies method arguments with `proceed(Object[] args)` causes a `ClassCastException` in some calls. How do you debug it?

Answer: The aspect is probably changing the argument array to a type that does not match the target method's parameter types, or the order of arguments is wrong. I start by logging the original argument types and the modified argument types inside the advice. I compare them to the method signature from the join point. The new arguments must be assignable to the corresponding parameter types, including primitive boxing and wrapper types. I also check whether the aspect is applied to overloaded methods, because the same modification may not be valid for every overload. If the modification is complex, I prefer moving the logic into the service method itself or using a dedicated decorator, because silently changing arguments through AOP can make method contracts hard to understand.

S29. A project uses AOP for declarative transaction management, and a new developer asks why a method is transactional even though it has no `@Transactional` annotation. How do you explain it?

Answer: I explain that Spring AOP applies advice based on pointcuts, not only on explicit annotations. A transaction aspect may be configured to match all methods in the service layer using an `execution(* com.example.service.*.*(..))` pointcut, or it may match a custom annotation that is placed at the class or interface level and inherited by methods. This is why the method appears transactional without its own annotation. I show the developer the aspect class and the pointcut expression so the behavior is visible. I also recommend adding a class-level annotation or an interface-level marker if the team wants to make the transactional boundary explicit, because hidden pointcuts can confuse new team members.

S30. A Spring Boot application starts slowly, and profiling shows that aspect initialization is taking a long time. How do you speed up startup?

Answer: Aspect initialization can be slow if pointcut expressions are very broad or if the application has many beans and aspects. I speed it up by narrowing pointcuts so Spring does not have to evaluate complex expressions against every bean. I also reduce the number of beans that are scanned by moving aspects and their target packages to smaller, focused component scans. If many aspects use the same broad pointcut, I combine them or refactor them into a single aspect. I also consider whether some aspects can be disabled in production if they are only needed for debugging. Finally, I check whether the aspect class uses `@Component` and is being eagerly initialized; if an aspect has heavy dependencies, I make them lazy or move the heavy work out of the aspect constructor.